Getting Started with Claude Code

A hands-on workshop

Alyssa Bilinski, PhD

Department of Health Services, Policy and Practice
Department of Biostatistics
Brown University

October 21, 2026

Accounts and downloads

- **Claude account:** Pro, Max, or Team plan, or an Enterprise seat with Claude Code (not the free plan)
- **GitHub account:** not your usual one, with nothing personal on it
- **Gmail account:** not your personal one, with nothing sensitive in it
- **Docker:** download Docker Desktop at docker.com/products/docker-desktop

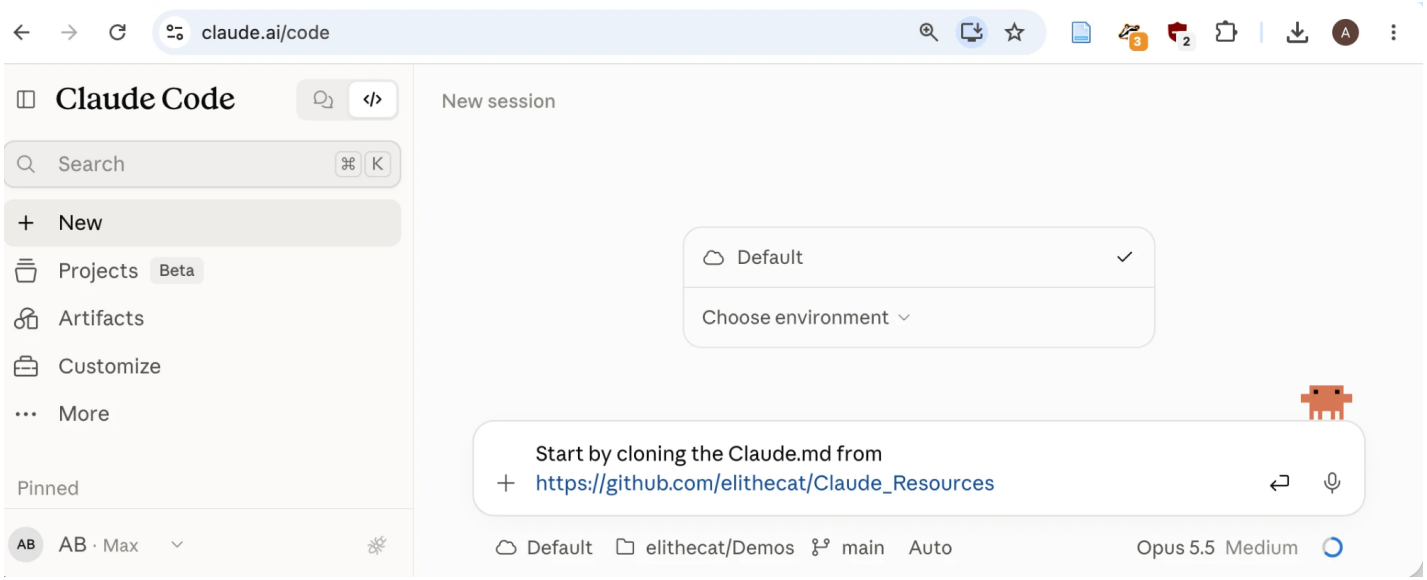
Three ways to run Claude Code

1. **In the cloud**
2. **On your computer**
3. **On a separate machine**

1

In the cloud

Claude Code in the cloud

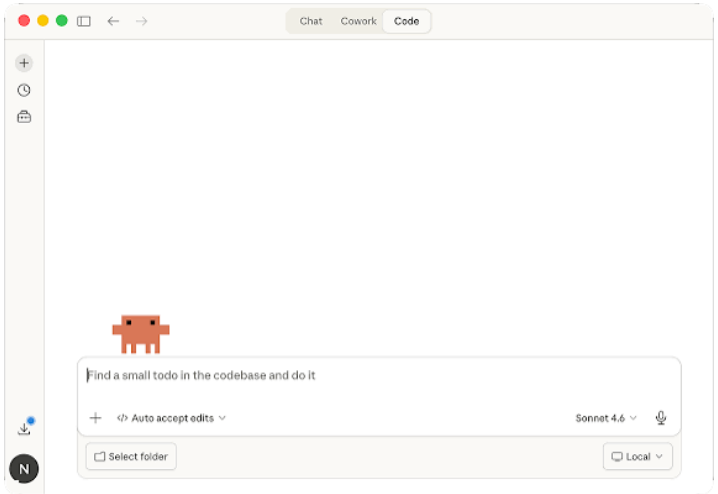


- Connect your GitHub account
- Pull a repository into a cloud session (new project: create an empty one first)
- Work within the repository
- Claude pushes its work to a new branch on GitHub
- See results in the session and on that branch

2

On your computer

Desktop app and command line



- In my experience, looks through a lot of your folders and asks for broad permissions
- Also available through the command line
- I wouldn't feel comfortable with this on my own computer, unless permissions are tight
- I suspect many of you wouldn't either

3

On a separate machine

Separate machine

- **Virtual machine:** Brown virtual machines (I've been trying to set this up, without much success yet)
- **Mac mini:** a desktop Mac without a screen, run from your own computer (I have one)
- **Container:** Docker, on your own computer

CLAUDE.md

Purpose of CLAUDE.md

- Plain-text instructions that Claude reads at the start of every session
- Explains a little about the project: what it is, where the files are, how to run it
- Project-specific rules
- General rules that you like, for every project
- **Best practice:** update it as you go. When Claude makes an error, add a rule

CLAUDE.md locations

- **Your rules, every project:** `~/.claude/CLAUDE.md` on your computer (cloud sessions don't read it)
- **Project rules, shared on GitHub:** `CLAUDE.md` in the project folder
- **Your notes on one project:** `CLAUDE.local.md`, added to `.gitignore`
- **Subfolders:** Claude loads a `CLAUDE.md` in a subfolder when it works on files there
- Claude combines all of them and reads the closest one last

Examples from my CLAUDE.md

- “Never lie, and never make up data, numbers, or facts.”
- “A question is not approval.”
- “When she asks you to fix one item, change only that item.”
- “Rebuild every affected output before you commit.”
- “Do not use em dashes.”

General template: github.com/elithecat/Claude_Resources

Getting started

Starting a project

- **Describe the project:** what it is and what it is for
- **Give context:** documents that will be useful
- **Specify the tasks:** what to do and what to deliver
- **Plan first:** ask for a plan before any changes (plan mode)
- **Give Claude a way to check its work:** tests or expected outputs

Models

- **Fable:** the most capable, for the hardest and longest tasks
- **Opus:** for complex reasoning
- **Sonnet:** for everyday coding tasks
- **Haiku:** fast, for simple tasks
- **Choose:** `/model`, for example `/model opus`

Depending on your plan, Fable can bill to usage credits instead of your plan's included limits.

Thinking level

- How much Claude reasons on each step ("effort")
- **Lower:** faster and cheaper, for straightforward tasks
- **Higher:** deeper reasoning, for complex problems
- **Levels:** low, medium, high, xhigh, max (Opus 5.5 defaults to medium)
- **Choose:** `/effort`, for example `/effort high`

My choice of model

- At least Opus 5.5 at high
- Sometimes Fable
- Never Sonnet

Version control

- Work in GitHub or another version control system
- See every change that Claude makes
- Revert the changes you don’t want

Plan mode

- Claude reads files and proposes a plan, without editing your files
- **Turn it on:** Plan in the mode menu (Shift+Tab in the terminal)
- Explore, then plan, then make changes, then commit
- Review and approve the plan before Claude edits files
- Skip it for small, clear changes

Intermediate output for review

- Ask for output at each step: tables, figures, and logs
- Review it in a rendered report (PDF)
- I'll show one of my reports

Subagents to check work

I like using subagents to check work.

- Separate agents review the work, each with one job
- **Yesterday's NIH committee comment:** 18 reviewers checked facts, voice, `CLAUDE.md` rules, signs of generated text, and formatting
- **Its reference list:** 4 subagents found an outdated title for a *JAMA* reference
- Each reviewer saves a report
- Tell each reviewer which files to check against

Commands and skills

Slash commands

- Type `/` at the start of a message to see the commands
- `/init`: write a starting `CLAUDE.md` for the project
- `/model opus` and `/effort high`: choose the model and the thinking level
- `/plan`: switch to plan mode before a large change
- `/help`: list all commands

Commands for long sessions

- **Context window:** the conversation and files that Claude can consider at once
- `/context`: show what fills the context window
- `/compact`: summarize the conversation to free space
- `/clear`: start a new conversation (in the cloud, start a new session instead)
- `/usage`: show your plan's usage limits and how much you have used

Skills

- Instructions in a `SKILL.md` file, with a description of when to use them
- Claude loads a skill only when it is relevant, or you run it with `/skill-name`
- Make one for a procedure that you keep pasting into chat
- **Your skills, every project:** `~/.claude/skills/` on your computer (cloud sessions don't read it)
- **Project skills, shared on GitHub:** `.claude/skills/` in the project folder

Skill examples

- `/skills`: list the skills you have
- **Built in:** `/code-review` checks your changes for bugs
- **Built in:** `/doctor` checks your setup and your `CLAUDE.md` files
- **One of mine:** `verify-figure` finds the primary source for a number and records where the number came from
- Ask Claude to turn a procedure from a session into a skill

Connectors and MCP

Connectors

- Let Claude use your other tools, for example Gmail and Google Drive
- Add them at claude.ai/customize/connectors, then sign in and grant access
- Available on personal plans, not yet on Brown's plan
- Available in Claude Code too (`/mcp` lists them)
- Connect the separate Gmail account, not your own

MCP servers

- **Model Context Protocol (MCP):** an open-source standard for connecting AI tools to other software
- Connectors are MCP servers
- **Directory:** claude.ai/directory
- **Add one in the terminal:**
  `claude mcp add --transport http <name> <url>`
- **For cloud sessions:** add `--scope project` and commit `.mcp.json`

Example projects

- **Make a user interface:** for example, a Shiny app to explore a model
- **Speed up code:** profile first, then check that the faster code gives the same results
- **Make slides from a paper:** start from my beamer template in Claude Resources
- **Reformat a paper for a new journal:** review every change on GitHub
- **Check a draft:** have subagents check the references and numbers

Plan for today

- **Most of today:** cloud sessions at claude.ai/code
- **Docker setup guide:** the claude-code-docker-setup folder in Claude Resources
- **Also new, in beta:** the Claude Science app for research, at claude.com/product/claude-science

alyssa_bilinski@brown.edu